

AquaScan AI+ Master Implementation Prompt

Single-paragraph, copy-ready brief for the complete engineering and modern realtime frontend upgrade.

Act as a senior full-stack AI, computer-vision, FastAPI, React/Vite, UX, accessibility, QA, and technical-documentation engineer, and upgrade the existing AquaScan AI project into AquaScan AI+ - Explainable Shadow-Aware Marine Debris Screening - without breaking any present capability. The project currently uses Python 3.12, an Ultralytics YOLOv8 model at `models/marine_debris_yolov8n_best.pt`, FastAPI in `src/api.py`, prediction logic in `src/predict.py`, optional localization in `src/localization.py`, PDF reporting in `src/report.py`, and a React/Vite frontend in `frontend/`. Treat this as a production-quality implementation request, not a prototype: inspect the complete repository first and provide a concise implementation plan before editing code; identify the current request and response structure of the prediction endpoint, how images are uploaded, stored, annotated, and returned, where detection results render in the frontend, the smallest safe set of modules needed for tiled inference, shadow-evidence analysis, structured explanation output, and richer PDF reporting, and the exact existing files to change and files to create. Preserve all working endpoints, model paths, output directories, current frontend behavior, command-line prediction behavior, and the existing upload-and-Detect workflow; do not delete, rename, or silently alter anything that currently works, do not use placeholder or fabricated production data, and implement each change in a backward-compatible, testable, documented manner. Add a `DETECTION_MODE` configuration setting with exactly baseline and enhanced values: baseline must preserve the existing YOLO inference behavior byte-for-byte in intent, enhanced may initially invoke the same inference path until the later enhanced modules are connected, the FastAPI prediction response must return `detection_mode`, the React dashboard must visibly show the active mode, errors must be clear without leaking server internals, `README.md` must explain switching modes, and the delivery must list every changed file and local test steps for both modes. Create `src/preprocessing.py` for conservative side-scan-sonar preprocessing with `preprocess_sonar_image(image: np.ndarray, enabled: bool = True) -> tuple[np.ndarray, dict]`; when enabled, perform only grayscale conversion when necessary, percentile-based contrast normalization, conservatively configured CLAHE, and optionally light median denoising, while preserving original dimensions, avoiding misleading color changes, never overwriting uploaded inputs, returning metadata for applied steps, parameters, original and processed shape, and enabled state, using a typed dataclass or config object with unit-testable helpers, controlled by `PREPROCESSING_ENABLED`, integrated only in enhanced mode, optionally saving a processed preview under `outputs/predictions/` only when explicitly enabled, returning preprocessing metadata through the API, and covered by synthetic grayscale-image tests. Create `src/tiled_inference.py` for enhanced-mode side-scan-sonar tiled inference using the existing YOLO model: split images into overlapping 640 x 640 tiles by default with 20 percent overlap on each axis, retain each tile's original full-image x/y offset, run the existing model on every tile, map every tile detection to valid full-image pixel coordinates, class-aware merge duplicate boxes using non-maximum suppression that retains the highest confidence, and return final full-image detections, raw pre-merge detections, tile count, tile size, overlap, NMS IoU threshold, elapsed inference time, and per-detection tile origin for debugging; support images smaller than one tile, preserve edge tiles without silent loss, prevent out-of-bounds boxes, write deterministic synthetic tests for coordinate mapping and NMS, use the module only when `DETECTION_MODE=enhanced`, retain baseline untouched, and expose the UI labels Small-target tiled analysis: enabled, tile count, and total inference time while explaining in `README` why tiling improves small-target detection. Create `src/shadow_analysis.py` for image-level highlight-and-shadow evidence, never a claim of physical object height unless sonar metadata is supplied and never automatic rejection or ground truth: take an original or conservatively preprocessed grayscale image, a full-image YOLO box, and optional typed configuration; extract a padded region of interest; calculate a 0-to-1 highlight score from relative brightness and local contrast in or near the object; find an adjacent dark, contiguous candidate shadow region; support expected shadow direction right, left, and auto; in auto, evaluate both sides, preserve the stronger evidence, and record the selected direction; calculate `highlight_score`, `shadow_darkness_score`, `shadow_continuity_score`, `shadow_proximity_score`, combined `shadow_evidence_score`, selected shadow direction, optional candidate shadow box, and an understandable reason; return only `SHADOW_SUPPORTED`, `REVIEW_REQUIRED`, or `INSUFFICIENT_EVIDENCE`; create an overlay with a yellow YOLO box, blue analyzed region, cyan candidate shadow box, status text, and scores; make all thresholds configurable in a typed dataclass; test synthetic bright-object-plus-adjacent-dark-region, no-nearby-dark-area, distant-dark-area, and border cases; document in code

and README that shadow direction and appearance depend on side-scan-sonar orientation, seabed, sonar settings, and acquisition geometry; run it after tiled inference only in enhanced mode and attach a structured shadow_analysis object to every detection response. Create src/anomaly_scoring.py as a lightweight explainable review-ranking module, not a confirmation classifier: for each YOLO detection and optionally non-detection image region, measure local intensity deviation from surrounding seabed, texture variation using a simple documented method such as local standard deviation, Laplacian variance, or GLCM only if dependencies remain manageable, and candidate-versus-surrounding-ring contrast; normalize and combine them into anomaly_score from 0 to 100, anomaly_level LOW, MEDIUM, or HIGH, component scores, a clear explanation, and a recommended_operator_action; never call an anomaly marine debris unless YOLO predicted a debris class, label high-anomaly/low-confidence detections Unusual region - manual review, label high-confidence detections with strong shadow evidence High-priority debris candidate, keep all thresholds in a typed config object, cover synthetic and edge-case tests, add the result to enhanced API output and PDF reports, and sort the React detection list in this order: High-priority debris candidate, Detected debris - review needed, then Unusual region - manual review. Create src/priority_scoring.py and transparently calculate for every enhanced final detection $\text{priority_score} = 0.55 * \text{yolo_confidence} + 0.30 * \text{shadow_evidence_score} + 0.15 * (\text{anomaly_score} / 100)$; return score from 0 to 1, level HIGH for score ≥ 0.70 , MEDIUM for 0.45 to less than 0.70, LOW below 0.45, the exact formula components, and a plain-language reason; clearly document that it is a triage and ranking score rather than ground-truth probability, retain raw YOLO confidence separately, show three component bars or percentages plus final priority in the frontend and explain it in the report, return null or not_applicable enhanced values in baseline rather than fabricating scores, and add threshold and formula unit tests. Refactor src/api.py with explicit Pydantic request and response models, automatic field descriptions in the API documentation, an api_version, and backward compatibility wherever feasible; every detection should validate fields equivalent to detection_id such as det_001, class_name such as marine_debris, confidence such as 0.84, bounding_box with x1, y1, x2, y2, width, and height, inference_source with mode, tile_id, and tile_origin x/y, shadow_analysis with status, highlight_score, shadow_darkness_score, shadow_continuity_score, shadow_proximity_score, shadow_evidence_score, shadow_direction, optional candidate_shadow_box, and reason, anomaly_analysis with anomaly_score, anomaly_level, and reason, priority with score, level, formula exactly $0.55 * \text{confidence} + 0.30 * \text{shadow_evidence} + 0.15 * (\text{anomaly_score} / 100)$, and reason, plus operator_recommendation; top-level response must include detection_mode, preprocessing metadata, inference timing, tiled-inference metadata, image dimensions, warnings and limitations, annotated output image URL or path according to current conventions, and report URL or path according to current conventions. Gracefully support no detections, retain useful safe error messages, write FastAPI TestClient endpoint tests, and put an example JSON response in README.md. Modernize only the relevant React/Vite frontend while retaining the existing upload and Detect objects workflow, VITE_API_URL configuration, and unrelated interface behavior: provide an easy, clearly labeled Baseline YOLO versus Enhanced small-target and shadow-aware analysis selector; show an at-a-glance responsive summary row for total detections, high-priority candidates, review-required candidates, total inference time, and enhanced tile count; include a high-quality annotated sonar image viewer with pan/zoom when practical, a priority-sorted accessible detection list, and a clickable detection card that highlights the selected image box, zooms or crops to its target, and shows confidence, shadow-evidence status, anomaly score, priority score, plain-language explanation, and operator recommendation. Use accessible semantic labels and visible text in addition to color, with green for shadow-supported/high-priority, amber for review-required, and gray or blue for insufficient evidence or unusual regions, and show the permanent notice: Shadow evidence is image-based decision support, not proof of object identity or physical height. Add a comparison experience when baseline and enhanced results are both available, displaying baseline detection count, enhanced detection count, targets newly discovered through tiled analysis, and runtime difference. Deliver a premium, modern, responsive, professional marine-operator interface with a restrained AquaScan visual language: a clean dark or light high-contrast design, consistent spacing, typography, cards, helpful empty/loading/error states, keyboard navigation, focus visibility, screen-reader labels, mobile and desktop layouts, and no low-contrast or color-only interactions. Add meaningful, performant micro-interactions and animations - skeleton/loading progress while an upload or analysis is running, animated mode transitions, subtle card hover and selection states, smoothly animated detection focus/overlay changes, number and progress-bar transitions that respect prefers-reduced-motion, and no distracting, long, or blocking effects. Build real-time data behavior from actual system events, not mock values: use a FastAPI WebSocket or Server-Sent Events endpoint, with safe polling fallback if the existing backend or deployment cannot maintain a persistent connection, to stream queued/uploading/processing/completed/failed analysis status, current stage, elapsed time, and result updates; reconnect safely, clean up subscriptions, prevent duplicate requests, retain a

usable non-real-time fallback, preserve all privacy and existing API conventions, and show clear connection/state labels in the UI. Add a compact real-time operations/analytics panel using accessible graphical charts, preferably with an established React chart library already present or a minimal, well-maintained addition: show actual data only, including live analysis progress/timeline, detection distribution by priority, shadow-evidence status distribution, anomaly-score bands, inference-time comparison between baseline and enhanced runs, tiled-analysis efficiency such as tile count versus runtime, and a time-filtered recent-analysis trend when historical results are available; support tooltips, text alternatives or data tables, keyboard-readable summaries, responsive resizing, empty states for insufficient data, and no chart that implies scientific certainty beyond the data. Keep chart data derived from validated API responses or a documented persisted history endpoint, retain UTC/timestamp handling transparently, document data retention and the absence of historical data when no store is configured, and expose safe typed schemas for any new real-time or history endpoints. Extend `src/report.py` so enhanced reports document explainable sonar-debris analysis and baseline reports continue working: create a unique non-overwriting PDF for each uploaded image with the title AquaScan AI+ - Explainable Shadow-Aware Marine Debris Screening, input filename, analysis timestamp, image size, selected detection mode, preprocessing and tiled-inference configuration, summary counts for total detections, high-priority candidates, review-required detections, unusual regions, and inference time, the full annotated image, and a results table with detection ID, class, YOLO confidence, bounding-box pixels, shadow status, shadow-evidence score, anomaly score, priority score, and operator action; provide a small crop or thumbnail for each high-priority detection with object and candidate-shadow visualization when available; include a limitations section stating that pixel coordinates are not geographic coordinates, shadow evidence is decision support only, physical height needs sonar metadata, and human verification is required before operational action; add a methods appendix covering tiled inference, duplicate-box merging, shadow-evidence screening, anomaly ranking, and the priority formula; handle empty detections without failure, test baseline and enhanced PDF generation, and update README instructions. Create a reproducible baseline-versus-enhanced evaluation pipeline in `src/evaluate.py` with output folders under `outputs/evaluation/` for baseline, enhanced, `review_queue`, `metrics_summary.csv`, and `experiment_report.md`; use existing YOLO data splits only if labels are present and never modify dataset labels; evaluate baseline full-image YOLO, enhanced preprocessing plus tiled inference plus duplicate merging, and enhanced output stratified by shadow-evidence status; calculate precision, recall, $mAP@0.5$, $mAP@0.5:0.95$ when feasible, F1, average inference time per image, number of detections, false positives, false negatives, and optional small-object metrics; define small object transparently as a configurable bounding-box area percentage of complete image area, such as 1 percent, without claiming it covers all real debris; never report shadow evidence as independent ground-truth accuracy unless manually labeled shadow-evidence data exists, instead summarize `SHADOW_SUPPORTED`, `REVIEW_REQUIRED`, and `INSUFFICIENT_EVIDENCE` as triage counts and proportions; export a manual-review CSV with image name, detection ID, crop path, confidence, shadow score, anomaly score, priority, and blank reviewer columns; use a fixed random seed where relevant, record model path and all configuration values, generate concise Markdown results tables and plain-language interpretation, and document exact Windows Command Prompt reproduction commands. Maintain the intended project layout with existing dataset and models folders, `outputs/predictions`, `outputs/reports`, `outputs/processed_previews`, `outputs/evaluation` with baseline, enhanced, `review_queue`, `metrics_summary.csv` and `experiment_report.md`, `src/api.py`, `src/predict.py`, `src/preprocessing.py`, `src/tiled_inference.py`, `src/shadow_analysis.py`, `src/anomaly_scoring.py`, `src/priority_scoring.py`, `src/evaluate.py`, `src/report.py`, `src/localization.py`, `src/tests/`, `frontend/`, `requirements.txt`, `README.md`, and the existing training notebook. Throughout implementation, prefer small composable modules, typed configuration, pure unit-testable functions, deterministic tests, clear file and path handling, no hidden global side effects, appropriate dependency pinning, centralized settings, safe validation, logging that assists local diagnosis without exposing secrets, meaningful user-facing errors, loading and no-result states, and robust behavior for grayscale, color, small, edge, and corrupted-image cases. At completion, provide a concise implementation summary, exact changed and newly created files, any migration or configuration values, exact local run and test commands for Windows, both baseline and enhanced verification steps, API and UI verification instructions including real-time behavior and charts, performance limitations, scientific limitations, a report-generation check, a reproducible evaluation example, and a short operator-oriented explanation of what the enhanced scores and visuals do and do not prove; do not claim performance gains, physical dimensions, autonomous decisions, live telemetry, or debris confirmation unless they are actually supported by measured data, available sonar metadata, and documented evaluation results.